

Walidacja dostępu do swojego (prywatnego) repozytorium na GitHub za pomocą kluczy i protokołu SSH.

Opis dotyczy systemu Windows, inne systemy – bez trudu da się znaleźć odpowiednie instrukcje w Internecie.

Generowanie kluczy SSH umożliwia bezpieczne uwierzytelnianie na serwerach, takich jak GitHub, bez użycia hasła. Klucz SSH składa się z dwóch części: **klucza publicznego**, który można udostępniać, oraz **klucza prywatnego**, który powinien pozostać zabezpieczony na komputerze użytkownika.

Aby korzystać z kluczy SSH, konieczne jest uruchomienie **SSH Daemon (sshd)**, który jest serwisem umożliwiającym nawiązywanie połączeń SSH na serwerze lub komputerze zdalnym. **sshd** nasłuchuje na odpowiednim porcie (domyślnie 22) i zarządza uwierzytelnianiem oraz szyfrowaniem połączeń.

Obsługa połączeń SSH: daemon sshd odbiera i akceptuje przychodzące połączenia SSH. Bez jego aktywacji komputer nie będzie w stanie przyjmować połączeń SSH, nawet jeśli klucze są poprawnie wygenerowane i skonfigurowane.

Aktywowanie sshd w Windows:

👉 https://learn.microsoft.com/en-us/windows-server/administration/openssh/openssh_keymanagement

Następnie można przejść czynności opisane tutaj:

👉 <https://docs.github.com/en/authentication/connecting-to-github-with-ssh/generating-a-new-ssh-key-and-adding-it-to-the-ssh-agent>

Generowanie klucza, przykład dla adresu twojmail@przyklad.pl. W terminalu PowerShell lub cmd:

```
> ssh-keygen -t ed25519 -C twojmail@przyklad.pl
```

Enter file in which to save the key (tu będzie katalog): można nadać swoją nazwę, np. id_mojkłucz

Kolejne kroki zatwierdzamy, passphrase jest dodatkowym zabezpieczeniem (hasłem), można pominąć – zatwierdzić puste.

Powstanie para plików (id_mojkłucz i id_mojkłucz.pub) we wskazanym katalogu, w szczególności:

```
c:\Users\twojekonto\.ssh\id_mojkłucz.pub
```

Zawartość tego pliku kopiujemy do schowka – można otworzyć w edytorze i skopiować – lub (będąc w tym katalogu):

```
> cat id_mojkłucz.pub | clip
```

Po **zalogowaniu się** na konto w GitHub (niech jego nazwa będzie dla przykładu: **twojekontogit**), klikamy w prawym górnym rogu i wybieramy **Settings**. Można też wpisać adres: <https://github.com/settings/profile>

Po lewej stronie klikamy: **SSH and GPG keys**

Klikamy w zielone pole **New SSH key**, otworzy się:

Add new SSH Key

Title

Key type

Authentication Key ↕

Key

Begins with 'ssh-rsa', 'ecdsa-sha2-nistp256', 'ecdsa-sha2-nistp384', 'ecdsa-sha2-nistp521', 'ssh-ed25519', 'sk-ecdsa-sha2-nistp256@openssh.com', or 'sk-ssh-ed25519@openssh.com'

W polu **Title** nadajemy dowolną nazwę, w polu **Key** wklejamy skopiowany do schowka klucz publiczny.

Zatwierdzamy (**Add SSH Key**).

I to by było wszystko, gdyby nie prawdopodobny fakt, że np. mamy w serwisie github.com więcej niż jedno konto, oraz na naszym komputerze mamy również kilka kluczy SSH, każdy zarejestrowany na innym z kont. Wtedy należy dopisać w pliku `c:\Users\twojekonto\.ssh\config`

```
Host etykieta1 # Konto pierwsze
  HostName github.com
  User git
  IdentityFile c:\Users\twojekonto\.ssh\id_rsa_main # Ścieżka do klucza SSH dla pierwszego konta
  IdentitiesOnly yes

Host kontoPython # wielkość liter ma znaczenie, inne konto
  HostName github.com
  User git
  IdentityFile c:\Users\twojekonto\.ssh\id_mojklucz # Ścieżka do klucza SSH dla drugiego konta
  IdentitiesOnly yes
```

Teraz można przetestować poprawność połączenia dla każdego z kont:

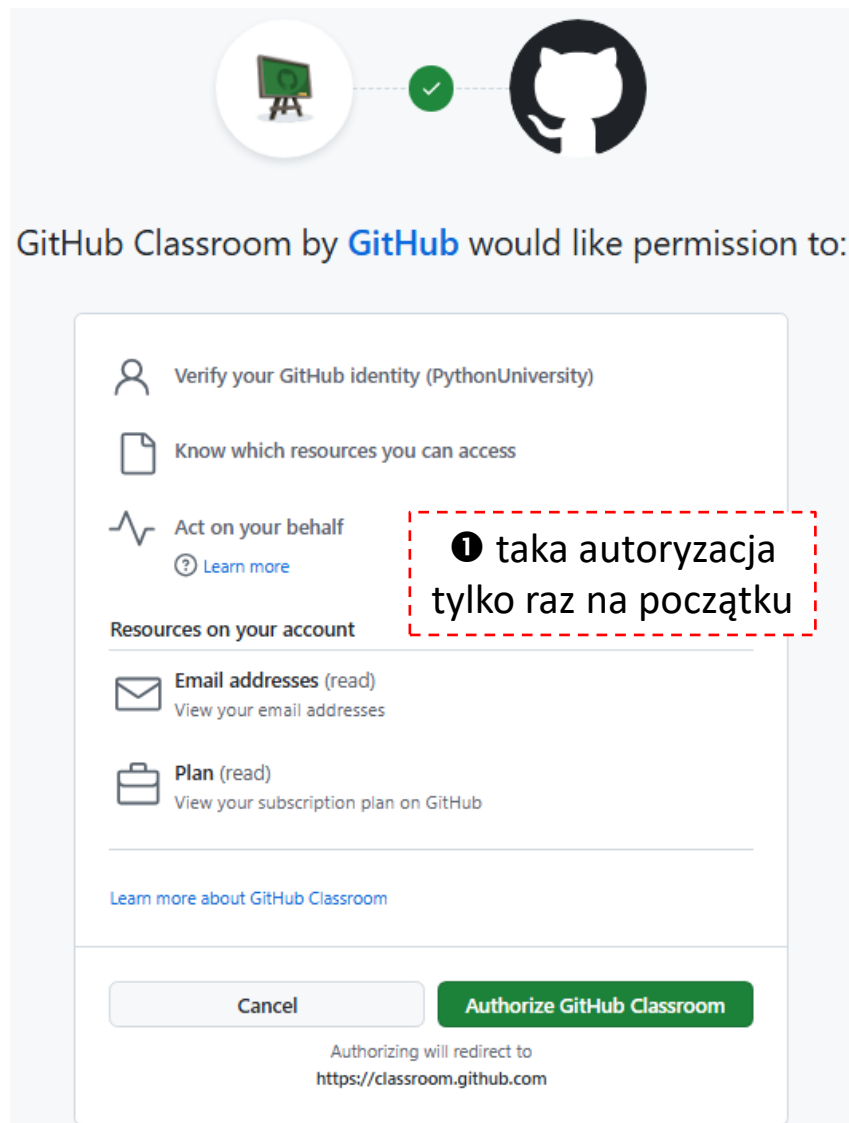
```
> ssh -T etykieta1
> ssh -T kontoPython
```

Przykładowa odpowiedź:

Hi **twojekontogit**! You've successfully authenticated, but GitHub does not provide shell access.

[aktualny zestaw testowy: **Zestaw 0**] – przyłącz się!

👉 <https://classroom.github.com/a/bSuev-Ky>



Jak działa GitHub Classroom i Assignment (zadanie)?

Każdy student otrzymuje swoje **własne repozytorium**, które jest kopią oryginalnego repozytorium bazowego. Repozytoria są prywatne, więc inne osoby nie mają do nich dostępu. W przeciwieństwie do typowego *forka*, repozytorium studenta nie ma bezpośredniego powiązania z repozytorium bazowym. **Nie jest traktowane jako *fork*** przez GitHub, więc nie można np. automatycznie tworzyć *pull requestów* do oryginalnego repozytorium bazowego. Ale – patrz slajd 11 – Classroom ma mechanizm pull request.

python

Accept the assignment — Zestaw 0

Once you accept this assignment, you will be granted access to the `zestaw-0-PythonUniversity` repository in the `wfais` organization on GitHub.

2 zaproszenie do zaakceptowania
Zestawu 0

Accept this assignment



You're ready to go!

You accepted the assignment, **Zestaw 0**.

Your assignment repository has been created:

🔗 <https://github.com/wfais/zestaw-0-PythonUniversity>

We've configured the repository associated with this assignment.

🔗 Open in GitHub Codespaces

📅 Your assignment is due by **Nov 10, 2024, 23:00 UTC**

3 kopia twojego repozytorium z **Zestawem 0** jest gotowa! Można się zająć rozwiązywaniem zadań.

"Fork" oznacza skopiowanie projektu (repozytorium) do własnego konta, aby móc nad nim pracować niezależnie od oryginalnej wersji. Można wtedy swobodnie wprowadzać zmiany bez wpływu na oryginalny projekt.

"Pull request" to prośba o włączenie wprowadzonych zmian z kopii projektu (forka) do głównego repozytorium. Daje twórcom oryginalnego projektu możliwość przeglądu i akceptacji zmian przed ich połączeniem.

Idziemy do swojej kopii repozytorium: <https://github.com/wfais/zestaw-0-PythonUniversity>

tu będzie oczywiście **twoja** nazwa

The screenshot shows the GitHub interface for a repository named 'zestaw-0-PythonUniversity', which is marked as 'Private'. It indicates it was forked from 'wfais/python-grupa4-zestaw-0-zestaw-0'. The main branch is 'master', and there are 2 branches and 0 tags. A search bar and buttons for 'Add file' and 'Code' are visible. A status bar shows 'This branch is 4 commits ahead of wfais/python-grupa4-zestaw-0-zestaw-0:master'. Below this, a commit by 'github-classroom[bot]' is shown with a green checkmark. A table lists files and folders: '.github', 'ZADANIE1', 'ZADANIE2', '.gitignore', and 'README.md', each with its commit history. The 'README' file is selected, showing a 'Review the assignment due date' button and an 'Open in GitHub Codespaces' button. The title 'Zestaw 0' is prominently displayed, followed by the description 'Repozytorium testowe do zadań na GitHub Classroom'.

File/Folder	Commit Message	Time Ago
.github	GitHub Classroom Feedback	3 minutes ago
ZADANIE1	Initial commit	25 minutes ago
ZADANIE2	Initial commit	25 minutes ago
.gitignore	Initial commit	25 minutes ago
README.md	add online IDE url; add deadline	2 minutes ago

Teraz możemy zacząć pracę!

- edytować kod w wewnętrznym edytorze, który jest uproszczoną wersją VS Code (Open in **GitHub Codespaces**)
 - zmiany trzeba będzie zatwierdzić w lokalnym repozytorium i wypchnąć do zewnętrznego, swojego repozytorium
- można edytować pliki wprost w swoim repozytorium (nie zalecane!)
- zrobić klona repozytorium na swój lokalny komputer i pracować z kodem lokalnie, używając dowolnego edytora, środowiska IDE (zalecane!)
 - zmiany trzeba będzie zatwierdzić w lokalnym repozytorium na dysku
 - następnie wysłać je do swojego zdalnego repozytorium

Edytowanie w GitHub Codespaces

zestaw-0-PythonUniversity-1 [Codespaces: fuzzy guacamole]

Review the assignment due date

Open in GitHub Codespaces

Imię i nazwisko:

Zestaw 0

Repozytorium testowe do zadań na GitHub Classroom

PROBLEMY DANE WYJŚCIOWE TERMINAL

@PythonUniversity → /workspaces/zestaw-0-PythonUniversity-1 (master) \$

- można zacząć od wpisania tu w terminalu **pytest**, aby się przekonać, że testy są gotowe i wykonują się po komendzie **pytest**
- możemy wejść do katalogu: **\$ cd ZADANIE1** i napisać **pytest** – ponownie zobaczymy test, ale tym razem tylko dotyczący pliku w katalogu ZADANIE1

CZAT

Copilot — Zapraszamy!

Zaczynajmy

Dodaj kontekst (#), rozszerzenia (@), polecenia

Kompiluj obszar roboczy

Pokaż konfigurację

Odpowiedzi sztucznej inteligencji mogą być niedokładne.

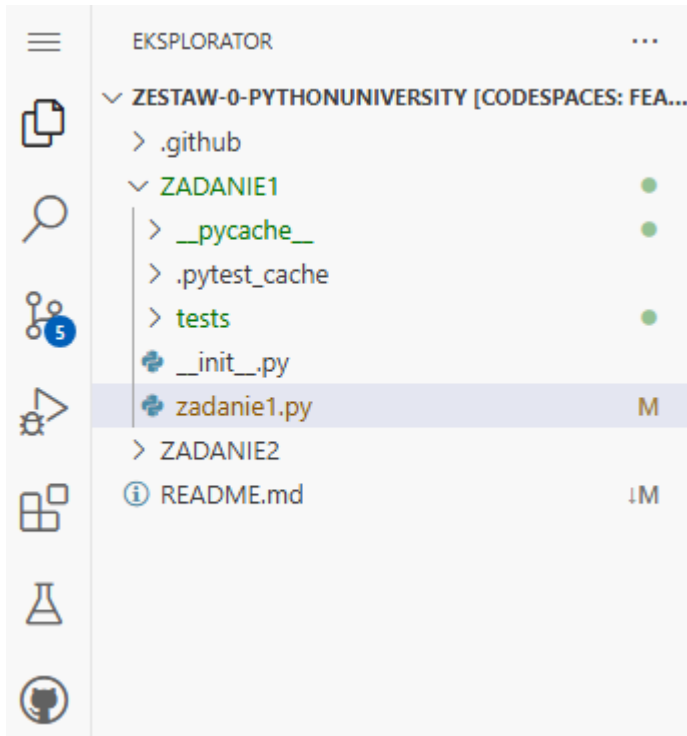
Zakończ konfigurację Układ: US

Otwórzmy plik *zadanie1.py* i uzupełnijmy o kod, który jest wymagany.

```

zadanie1.py 1 x
ZADANIE1 > zadanie1.py > permutacja
1 def permutacja(x, y, z, n):
2     pass # return lista list
3
4 if __name__ == '__main__':
5     x, y, z, n = # wczytaj dane
6     lista = permutacja(x, y, z, n)
7     print(lista)

```



W każdej chwili możemy uruchomić nasz program:

```

@PythonUniversity → /workspaces/zestaw-0-PythonUniversity/ZADANIE1 (main) $ python zadanie1.py
Podaj dane wejściowe x y z n, oddzielone spacjami: 1 1 2 3
[[0, 0, 0], [0, 0, 1], [0, 0, 2], [0, 1, 0], [0, 1, 1], [1, 0, 0], [1, 0, 1], [1, 1, 0], [1, 1, 2]]

```

Możemy program sprawdzić testami:

```

@PythonUniversity → /workspaces/zestaw-0-PythonUniversity/ZADANIE1 (main) $ pytest
===== test session starts =====
platform linux -- Python 3.12.1, pytest-8.3.3, pluggy-1.5.0
rootdir: /workspaces/zestaw-0-PythonUniversity/ZADANIE1
plugins: anyio-4.6.0
collected 6 items

tests/test_zadanie1.py .....

===== 6 passed in 0.01s =====
@PythonUniversity → /workspaces/zestaw-0-PythonUniversity/ZADANIE1 (main) $

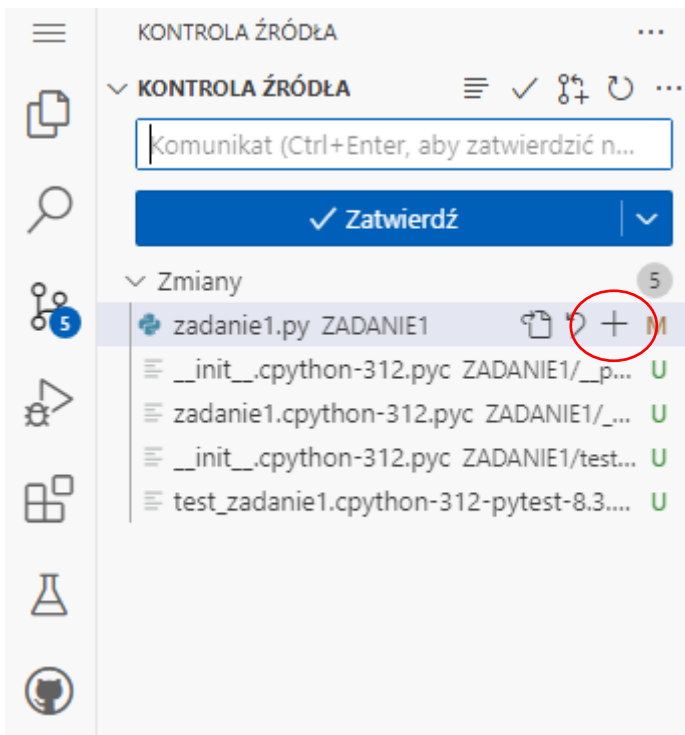
```

taki wynik jest
oczekiwany!

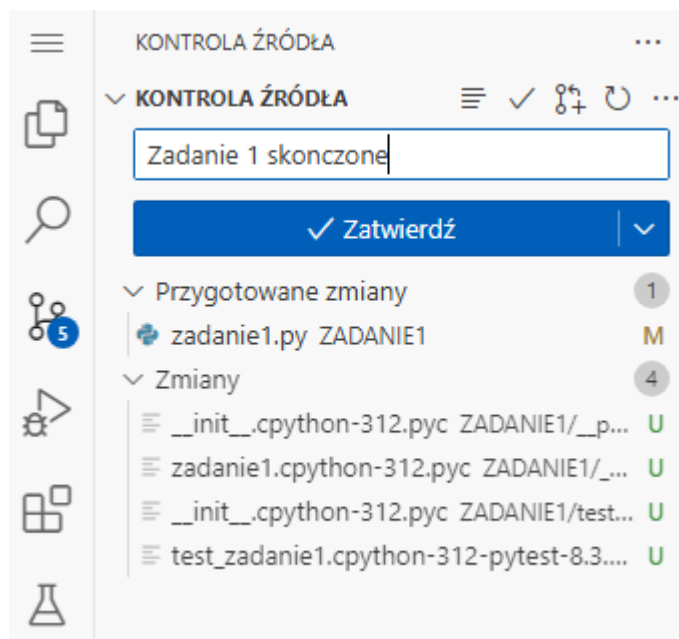
- jeśli testy wskazują, że nasz program jest poprawny, albo kiedy chcemy zachować plik, możemy przejść do etapu dołączenia kodu do swojego repozytorium
- znaczek M oznacza plik zmodyfikowany (ang. **modified**)
- **UWAGA:** w tym momencie nasz plik *zadanie1.py* nie jest jeszcze zachowany w repozytorium, ale lokalnie jest utrwalony (po ponownym otwarciu edytora GitHub Codespaces zobaczymy wprowadzone zmiany)
- klikamy ikonę Kontrola źródła  lub skrót klawiszowy Ctrl-Shift G

Wyślemy teraz *zadanie1.py* do **lokalnego** repozytorium środowiska GitHub Codespaces

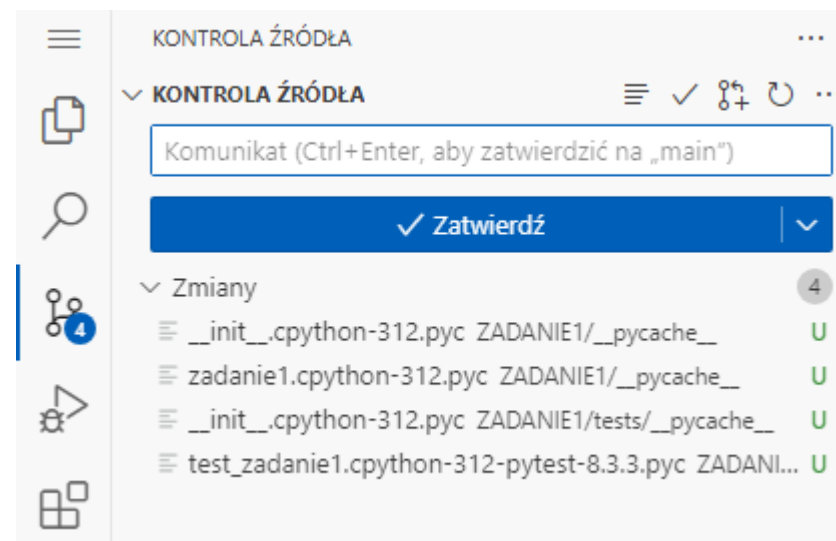
❶ klikamy znaczek + w linii *zadanie1.py*, jest to wysłanie pliku do poczekalni (Stage Changes)



❷ wpisujemy tekst opisujący zmiany wysyłane do repozytorium (tutaj tekst "Zadanie 1 skończone"), zatwierdzamy



❸ po zatwierdzeniu, plik *zadanie1.py* znika z pola "Zmiany", liczba plików zmienionych bądź nieśledzonych zmniejsza się o jeden. Plik jest w **lokalnym** repozytorium.



⇒ **WAŻNE!** Lokalne repozytorium w środowisku GitHub Codespaces **jest dostępne tylko w ramach tego środowiska**. Lokalna kopia repozytorium istnieje na wirtualnej maszynie Codespaces i nie jest dostępna bezpośrednio z zewnątrz, dopóki nie zrobisz push do twojego zdalnego repozytorium na GitHubie.

Lokalnie (np. w terminalu u dołu strony) można oczywiście wykonać komendy z repozytorium, np. `git status`, `git log`, itd.

Wyślemy teraz zmiany z naszego **lokalnego** repozytorium w Codespaces do naszego **zdalnego** repozytorium GitHub. W naszym przykładzie jest to repozytorium pod adresem <https://github.com/wfais/zestaw-0-PythonUniversity>. Można łatwo sprawdzić np. wpisując w terminalu: **git remote -v**, etykiety i przypisane im adresy repozytoriów.

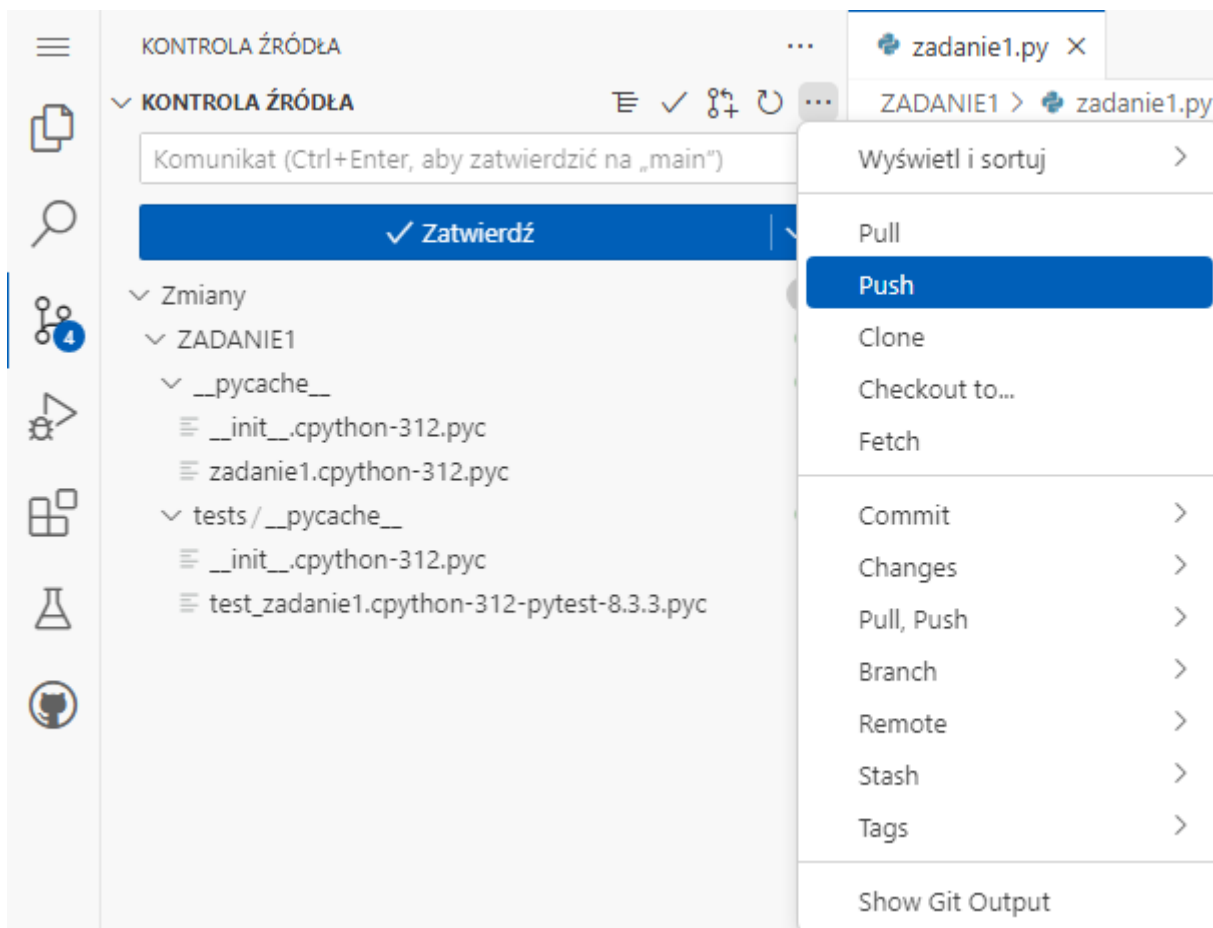
```
@PythonUniversity → /workspaces/zestaw-0-PythonUniversity (main) $ ls
README.md ZADANIE1 ZADANIE2
@PythonUniversity → /workspaces/zestaw-0-PythonUniversity (main) $ git remote -v
origin https://github.com/wfais/zestaw-0-PythonUniversity (fetch)
origin https://github.com/wfais/zestaw-0-PythonUniversity (push)
upstream https://github.com/wfais/python-zestaw-0-grupa-4-1.git (fetch)
upstream https://github.com/wfais/python-zestaw-0-grupa-4-1.git (push)
@PythonUniversity → /workspaces/zestaw-0-PythonUniversity (main) $
```

Podsumujmy dotychczasowe kroki!

W GitHub Codespaces oraz w każdym innym środowisku korzystającym z Git, wykonany proces można opisać następująco:

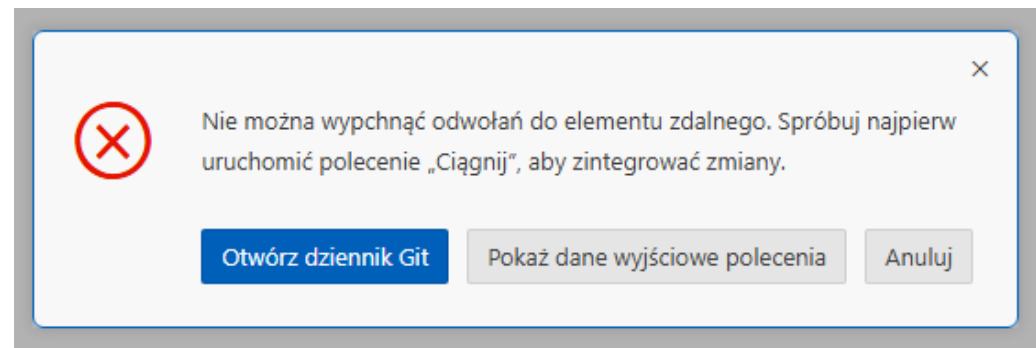
- **Dodanie do "Stage" (Stage Changes):** Kiedy klikasz znak "+" przy pliku *zadanie1.py*, przenosisz ten plik do sekcji "staged" – czyli przygotowujesz go do zatwierdzenia. Zmiany w tym pliku są teraz oznaczone jako gotowe do dodania do historii repozytorium, ale jeszcze nie są zapisane w repozytorium. Odpowiada temu wydanie komendy: **git add** .
w terminalu (kropka . to wszystkie pliki, więc lepiej – żeby **nie** wysyłać plików tymczasowych, binarnych, przynajmniej dopóki ich nie wykluczymy wpisując do pliku `.gitignore` – **bezpieczniej** jest napisać: **git add zadanie1.py**)
- **Zatwierdzenie zmian (Commit):** Po wpisaniu komunikatu (np. "Zadanie 1 zakończone") i kliknięciu "Zatwierdź" (Commit), tworzysz nową wersję (commit) w lokalnym repozytorium na Codespaces. Odpowiada temu wydanie komendy: **git commit -m "Zadanie 1 zakończone"** w terminalu. Zmiana jest teraz zapisana jako commit lokalnie, ale nie została jeszcze wysłana do twojego głównego repozytorium na GitHubie.

- **Push do repozytorium na GitHubie:** aby zobaczyć zmiany w repozytorium na GitHubie, musisz je jeszcze wypchnąć (push). Możesz to zrobić z Codespaces, wybierając opcję "Wypchnij" (Push). Odpowiada temu wydanie komendy: **git push** w terminalu. Po tej operacji zmiany (commity) trafią do zdalnego repozytorium na GitHubie i staną się widoczne, również dla prowadzącego kurs. Operacje te można wykonywać wielokrotnie, np. wysyłając poprawkę, albo kolejne pliki z zadaniami.



Co zrobić jeśli coś pójdzie nie tak?

Na przykład, wyskoczy takie okienko:



Komunikat oznacza, że przed wykonaniem `git push` musisz najpierw pobrać (pull) zmiany ze zdalnego repozytorium. Dzieje się tak, gdy w repozytorium zdalnym są nowe commity, których nie masz w swojej lokalnej kopii, i Git nie pozwala na push bez wcześniejszej synchronizacji tych zmian. Pull może się jednak również nie udać, bo Git będzie pytał o strategię łączenia zmian. Wyjaśnienie tego wymaga dłuższego opowiadania – i bardziej pasuje do kursu Git, dlatego tutaj proponujemy doraźne rozwiązanie. **Anuluj** powyższy komunikat. W terminalu wydaj komendę:

`$ git pull --rebase`

Następnie można powtórzyć push. Może to być też kolejna komenda w terminalu:

`$ git push`

➡ Zobaczmy, co się dzieje po stronie GitHub Classroom, ale również w repozytorium, gdy wyślemy (wypchniemy) zadanie.

przed wysłaniem:

Total students


PythonUniversity
Not submitted

@PythonUniversity
 0 commits

po wysłaniu:

Total students


PythonUniversity
Submitted

@PythonUniversity
 Latest commit 1 minute ago
1 commit
6.428571428571429/10

- Instruktor widzi w repozytorium na GitHubie, że uczestnik **PythonUniversity** wysłał jakieś zmiany do repozytorium.
- Symbol ✖ oznacza, że któryś z testów nie zakończył się pozytywnie. W tym przypadku to naturalne, bo wysłaliśmy do repozytorium dopiero rozwiązanie pierwszego zadania.
- Liczba 6.428571428571429/10 pochodzi z systemu automatycznej punktacji i zależy od konfiguracji testów, ich liczby i maksymalnej liczby punktów do uzyskania (tutaj 10).
- Instruktor może teraz podejrzeć, które dokładnie testy zakończyły się pozytywnie, a które nie.

Total points for zestaw-0-testy: 6.43/10

Test runner summary

Test Runner Name	Test Score	Max Score
zestaw-0-testy	6.42857142	10
Total:	6.42857142	10

🏆 Grand total tests passed: 9/14

Error: Some tests failed.

Oczywiście, ma wgląd na zmiany wykonane w kodzie, np. klikając w tekst (komentarz) ostatniego commitu:

zestaw-0-PythonUniversity Private

Your main branch isn't protected

This branch is 5 commits ahead of main

PythonUniversity committed 42 minutes ago

Zadanie 1 skonczone

29b3a8f · 41 minutes ago · 7 Commits

.github	GitHub Classroom Feedback	2 hours ago
ZADANIE1	Zadanie 1 skonczone	41 minutes ago
ZADANIE2	Initial commit	2 hours ago
README.md	add online IDE url; add deadline	2 hours ago

1 file changed +2 -2 lines changed

ZADANIE1/zadanie1.py

```

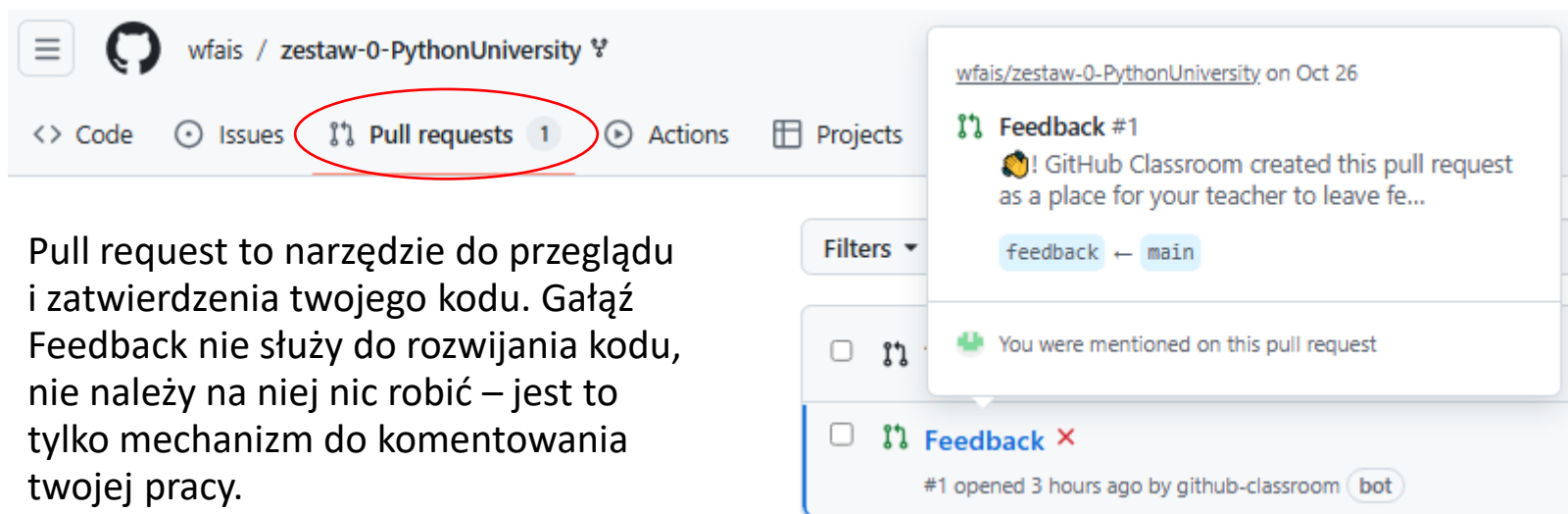
@@ -1,7 +1,7 @@
1 1 def permutacja(x, y, z, n):
2 - pass # return lista list
2 + return [[i, j, k] for i in range(x + 1) for j in range(y + 1) for k in range(z + 1) if i + j + k != n]
3
4 4 if __name__ == '__main__':
5 - x, y, z, n = # wczytaj dane
5 + x, y, z, n = map(int, input("Podaj dane wejsciowe x y z n, oddzielone spacjami: ").split())
6 6 lista = permutacja(x, y, z, n)
7 7 print(lista)

```

Dokładnie taki sam podgląd ma właściciel repozytorium, użytkownik **PythonUniversity**! Wybierając pole "Actions" ma też podgląd na raport:

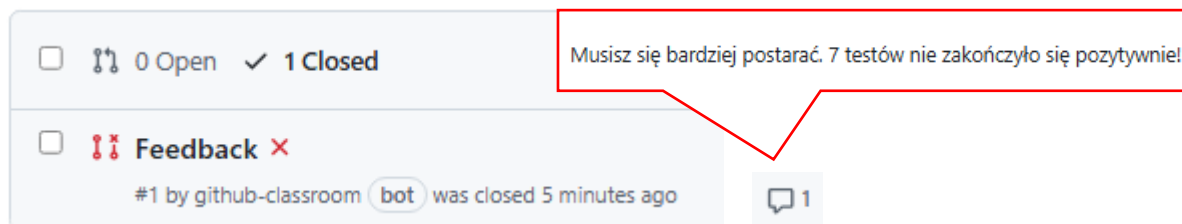
✖ run-autograding-tests
Some tests failed.

- Po zaakceptowaniu zadania (Assignment), oprócz kopii repozytorium (gałąź **main**) tworzona jest dodatkowa gałąź, nazwana **Feedback**.
[oczywiście, jeśli tak zostało skonfigurowane]
- Jest to związane z "Pull requests", które pojawia się na górnym pasku:



Bot GitHub Classroom automatycznie otwiera **pull request**, aby ułatwić proces przeglądu i oceny twojego rozwiązania.

Gałąź **Feedback** powstaje, aby umożliwić przegląd i dodawanie komentarzy bez modyfikacji twojej głównej pracy. Zatem wszelkie komentarze do rozwiązania zadania, jeśli w ogóle, pojawią się właśnie w ramach pull request. Zakończenie przeglądu danego rozwiązania będzie wiązało się z zamknięciem pull request, oczywiście bez scalania kodu z kodem źródłowym głównego repozytorium.



⇒ Teraz zobaczymy, w jaki sposób można edytować zadanie poza GitHub Codespaces, czyli albo **wprost w repozytorium**, albo jeszcze lepiej, wykonując kopię (clone) repozytorium na nasz lokalny dysk. Wtedy możemy użyć dowolny edytor/IDE.

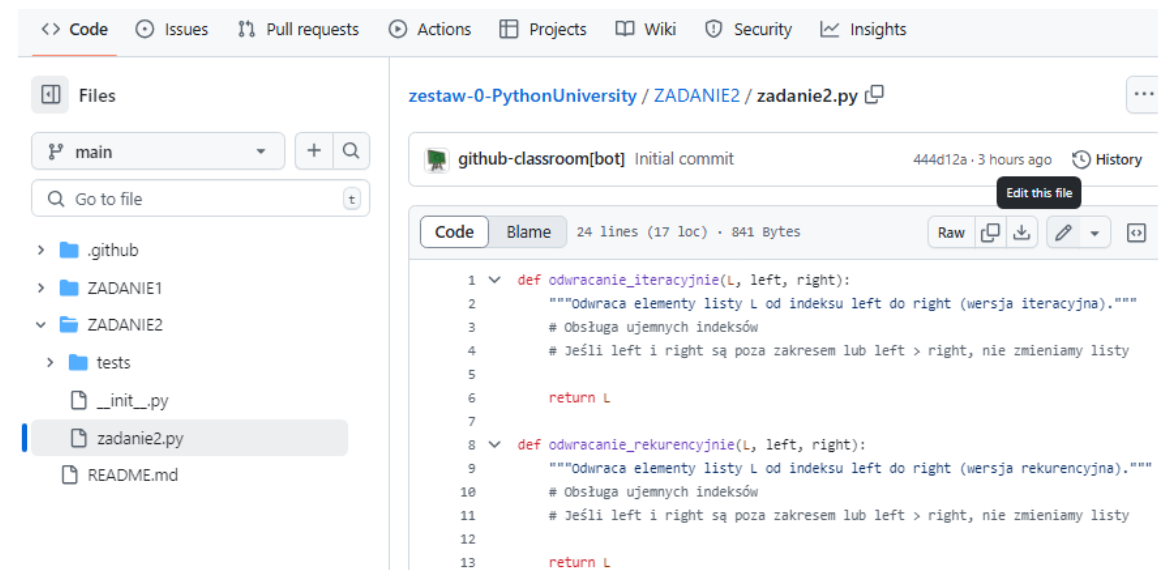
W zasadzie, edycja kodu bezpośrednio w repozytorium jest najprostsza i najszybsza, ale pozbawieni jesteśmy wtedy zalet środowiska (edytora) wspierającego tworzenie kodu. Nie możemy też kodu uruchomić.

Zatem taki sposób pracy nie jest zalecany.

Edycję kodu kończymy zatwierdzając (zielony) przycisk "Commit changes"

Cancel changes Commit changes...

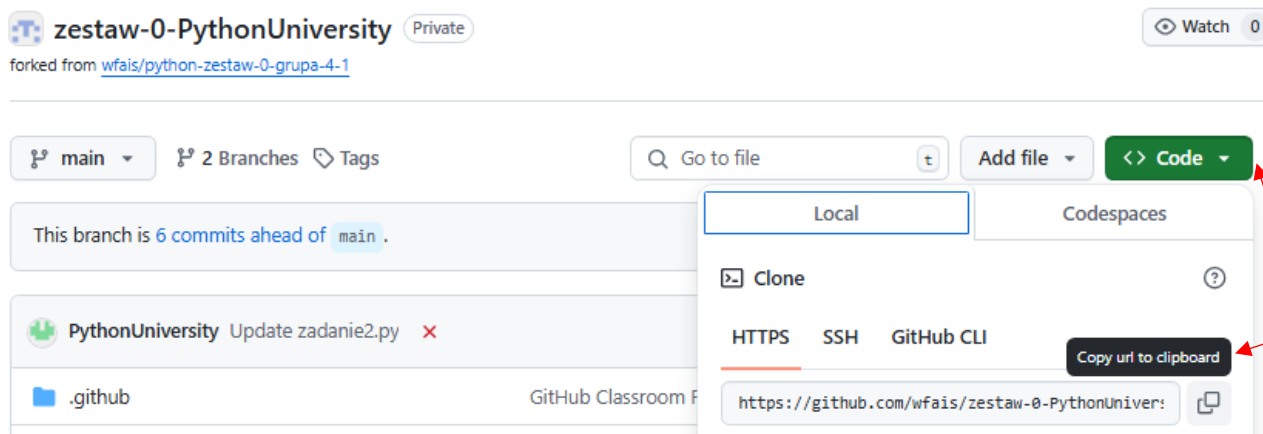
(wpisując komentarz do tego commitu). Taka zmiana również uruchamia automatyczne testowanie.



Klonowanie repozytorium (<https://github.com/wfais/zestaw-0-PythonUniversity>)

Repozytorium **publiczne**, można pobrać (sklonować) na swój dysk prostą instrukcją (terminal):

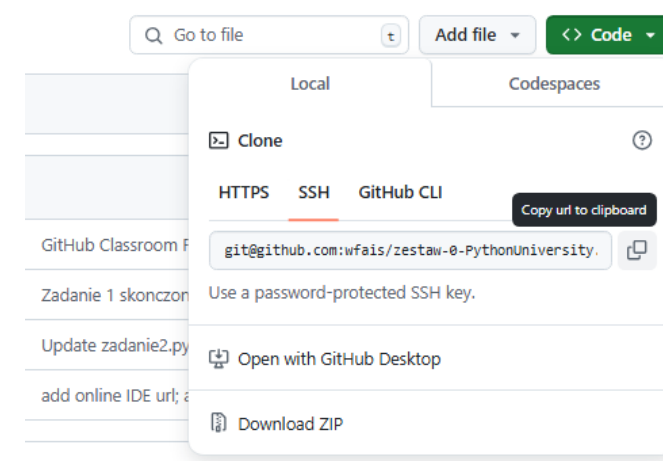
```
> git clone https://github.com/wfais/zestaw-0-PythonUniversity.git
```



```
> git clone kontoPython:wfais/zestaw-0-PythonUniversity.git
```

[to jest polecany sposób]

nie musimy przepisywać tego długiego adresu, można go skopiować



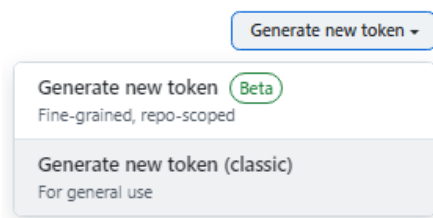
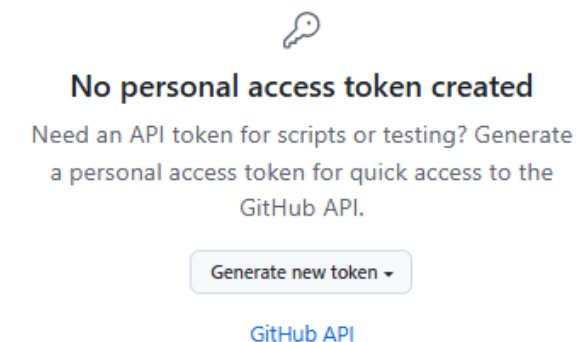
Jednak nasze repozytoria są **prywatne**, wymagają jakieś formy uwierzytelnienia. Rozważmy HTTPS i SSH.

- HTTPS** – wymagany jest token (PAT – Personal Access Token), wtedy instrukcja będzie miała postać:

```
> git clone https://<uzytkownik>:<token>@github.com/wfais/zestaw-0-PythonUniversity.git
```

Generowanie tokena: <https://github.com/settings/tokens>

Procedura nie wymaga większego komentarza, wybierzmy Tokens (classic)



Możemy zostać poproszeni o wpisanie hasła do konta. ...a potem zobaczymy multum opcji do wyboru!

New personal access token (classic)

Personal access tokens (classic) function like ordinary OAuth access tokens. They can be used instead of a password for Git over HTTPS, or can be used to [authenticate to the API over Basic Authentication](#).

Note
Klonowanie repozytorium

What's this token for?

Expiration *

30 days

The token will expire on Mon, Nov 25 2024

[na dole strony]

Generate token

Select scopes

Scopes define the access for personal tokens. [Read more about OAuth scopes.](#)

☒ repo	Full control of private repositories
☐ repostatus	Access commit status
☐ repo_deployment	Access deployment status
☐ public_repo	Access public repositories
☐ repo_invite	Access repository invitations
☐ security_events	Read and write security events

Token widać tylko raz! Oczywiście, można go też od razu usunąć / wygenerować nowy itd.

Personal access tokens (classic)

Tokens you have generated that can be used to access the [GitHub API](#).

Make sure to copy your personal access token now. You won't be able to see it again!

ghp_j1vvIEEnX2v64pqUpwdjQmbGV1QgsKf38uHjH

Posiadamy sklonowane repozytorium, czyli również kod zadań do wykonania. W przykładzie należy uzupełnić kod ZADANIE2.

```
def odwracanie_iteracyjnie(L, left, right):
    """Odwraca elementy listy L od indeksu left do right (wersja iteracyjna)."""
    # Obsługa ujemnych indeksów
    # Jeśli left i right są poza zakresem lub left > right, nie zmieniamy listy

    return L

def odwracanie_rekurencyjnie(L, left, right):
    """Odwraca elementy listy L od indeksu left do right (wersja rekurencyjna)."""
    # Obsługa ujemnych indeksów
    # Jeśli left i right są poza zakresem lub left > right, nie zmieniamy listy

    return L
```

Należy uzupełnić kod dwóch funkcji (jak po lewej stronie).

Mamy również pełny dostęp do wszystkich testów (katalog tests).

Co ciekawe, kod, w którym nic nie zmieniliśmy, przechodzi niektóre testy.

```
===== test session starts =====
platform win32 -- Python 3.12.7, pytest-8.2.0, pluggy-1.5.0
rootdir: C:\tmp\PYTHON\2024\GR4\ZESTAW0\repo\zestaw-0-PythonUniversity\ZADANIE2
plugins: anyio-4.2.0
collected 8 items

tests\test_zadanie2.py ..FFFF.F
```

Założmy, że mamy w pliku **ZADANIE2/zadanie2.py** już taki kod, który przechodzi 100% testów i chcemy go wysłać.

W tym miejscu zaznaczam, że **nie** prezentujemy tu kursu repozytorium Git, tylko absolutne minimum czynności, żeby odpowiedni kod poprawnie został wysłany do repozytorium.

```
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   ZADANIE2/zadanie2.py
```

Komenda: `> git status`

pokazuje, że plik **zadanie2.py** rzeczywiście jest zmieniony.

Wysyłamy zmianę do lokalnego repozytorium (tego skopiowanego na nasz dysk).

`> git add ZADANIE2/zadanie2.py` # sprawdź teraz git status

`> git commit -m "Zadanie 2 skonczone"`

`> #` warto sprawdzić, czy zdalne repozytorium jest dobrze ustawione, ale tak być powinno, jeśli je pobraliśmy zgodnie

`> #` z opisem z poprzedniego slajdu: wtedy etykieta zdalnego repozytorium (domyślna nazwa) będzie origin

`> git push` # wysłanie zmian zatwierdzonych w lokalnym repozytorium do zdalnego repozytorium

Assignment Details

Accepted assignments 1
1 Students

Assignment submissions 1
1 Submitted 0 Not submitted

Passed students 1
1/1 Passed

Filters Search for an assignment Filter by passing Sort

Total students

PythonUniversity Submitted
@PythonUniversity Latest commit 2 minutes ago ✓ 3 commits 9.999999999999999/10

Repository Feedback

Test runner summary

Test Runner Name	Test Score	Max Score
zestaw-0-testy	10	10
Total:	10	10

🏆 Grand total tests passed: 14/14

Na tym kończy się proces wysyłania zadań z określonego zestawu. Pamiętajmy, poprawek można robić dowolnie dużo, wysyłając kolejne zmiany do repozytorium.